VIT Vidyalankar Institute of Technology ACCREDITED A+ BY NAAC	**Department of Computer Engineering**
Semester	T.E. Semester V – CMPN
Division & Batch	A & Batch 1
Subject	Artificial Intelligence Lab
Laboratory Teacher:	Mahesh Khandke
Laboratory	

Student Name	Satyam Revgade	
Roll Number	24102A0035	
Grade and Subject Teacher's Signature		

Experiment Number	P8
Experiment Title	Wumpus World Problem using Knowledge-Based Reasoning
Problem Statement	The Wumpus World is a grid-based environment in which an intelligent agent must explore the world, find the gold, avoid dangerous pits, and avoid or kill the Wumpus. The agent does not have complete information about the environment; it only receives percepts from the current cell and must use logical reasoning to determine which neighbouring cells are safe. Your task: design and implement a knowledge-based Wumpus World agent that uses percepts and logical inference to identify safe cells, locate the Wumpus and gold, and generate a safe path to the gold, using Python. **Percepts used by the agent:** Breeze – a pit exists in an adjacent cell Stench – the Wumpus exists in an adjacent cell Glitter – gold is present in the current cell Bump – the agent has hit a wall Scream – the Wumpus has been killed **Problem Setup:**

	Grid size: 4 × 4 Agent start cell: (1,1) — coordinates written as (row, column) Goal: reach the cell containing Glitter and grab the gold Percepts obtained as the agent explores: (1,1): No Breeze, No Stench (1,2): Breeze (2,1): Breeze (2,2): No Breeze, No Stench (2,3): Stench (3,2): Stench (4,2): No Breeze, No Stench (4,3): Glitter **Logical Rules:** Rule 1 (Breeze): Breeze(x,y) → Pit(one adjacent cell) Rule 2 (No Breeze): ¬Breeze(x,y) → no adjacent cell has a pit Rule 3 (Stench): Stench(x,y) → Wumpus(one adjacent cell) Rule 4 (No Stench): ¬Stench(x,y) → no adjacent cell has the Wumpus Rule 5 (Safe): Safe(x,y) ↔ ¬Pit(x,y) ∧ ¬Wumpus(x,y) **Task 1 – Knowledge Representation:** Cell (2,2) is safe: Safe(2,2) Cell (3,3) contains the Wumpus: Wumpus(3,3) Cell (1,3) contains a pit: Pit(1,3) If a cell has breeze, an adjacent cell contains a pit: ∀x,y Breeze(x,y) → [Pit(x-1,y) ∨ Pit(x+1,y) ∨ Pit(x,y-1) ∨ Pit(x,y+1)] If a cell has stench, an adjacent cell contains the Wumpus: ∀x,y Stench(x,y) → [Wumpus(x-1,y) ∨ Wumpus(x+1,y) ∨ Wumpus(x,y-1) ∨ Wumpus(x,y+1)] **Task 2 – Inference:** For every percept the agent updates three sets: SAFE cells (no pit and no Wumpus), POSSIBLE-PIT cells (from a breeze whose source is not yet fully explained), and POSSIBLE/CONFIRMED-WUMPUS cells (from a stench). A candidate becomes CONFIRMED once every other neighbour of its stench source has independently been shown safe — this is how the agent locates the Wumpus at (3,3) without ever stepping into that cell. **Task 3 – Agent Architecture:** Environment → Sensors → Percepts → Knowledge Base → Inference Engine → Decision Making → Action → Environment

Resources / Apparatus Required	Hardware: Personal Computer / Laptop	Software: Python 3 (Jupyter Notebook / VS Code)

Code

```
"""
Wumpus World - Knowledge Based Agent
Grid: 4 x 4, coordinates written as (row, column)

Rules implemented:
  Rule 1  Breeze(x,y)   -> Pit(one adjacent cell)
  Rule 2 ~Breeze(x,y)   -> no adjacent cell has a pit
  Rule 3  Stench(x,y)   -> Wumpus(one adjacent cell)
  Rule 4 ~Stench(x,y)   -> no adjacent cell has the Wumpus
  Rule 5  Safe(x,y) <-> ~Pit(x,y) AND ~Wumpus(x,y)
"""

GRID_SIZE = 4

# Percepts as given in the problem statement (Section 4)
PERCEPTS = {
    (1, 1): {"breeze": False, "stench": False, "glitter": False},
    (1, 2): {"breeze": True,  "stench": False, "glitter": False},
    (2, 1): {"breeze": True,  "stench": False, "glitter": False},
    (2, 2): {"breeze": False, "stench": False, "glitter": False},
    (2, 3): {"breeze": False, "stench": True,  "glitter": False},
    (3, 2): {"breeze": False, "stench": True,  "glitter": False},
    (4, 2): {"breeze": False, "stench": False, "glitter": False},
    # Breeze/Stench not reported for (4,3) in the problem statement (only
    # "Glitter" is given for this cell) -> treated as "not observed" so it
    # does not conflict with the Wumpus already confirmed at (3,3).
    (4, 3): {"breeze": None,  "stench": None,  "glitter": True},
}

# Exploration order actually followed by the agent (Section 6)
PATH = [(1, 1), (2, 1), (2, 2), (3, 2), (4, 2), (4, 3)]


def neighbors(cell):
    x, y = cell
    candidates = [(x - 1, y), (x + 1, y), (x, y - 1), (x, y + 1)]
    return [c for c in candidates if 1 <= c[0] <= GRID_SIZE and 1 <= c[1] <=
GRID_SIZE]


class KnowledgeBase:
    """Everything the agent currently believes about the world."""

    def __init__(self):
        self.visited = {(1, 1)}
        self.no_pit = {(1, 1)}
        self.no_wumpus = {(1, 1)}
        self.possible_pit = set()
        self.possible_wumpus = set()
        self.confirmed_wumpus = set()
        self.stench_sources = []

    def apply_percept(self, cell, percept):
        before_safe = set(self.safe_cells())
        self.visited.add(cell)
        self.no_pit.add(cell)
        self.no_wumpus.add(cell)
        self.possible_pit.discard(cell)
```

```
        self.possible_wumpus.discard(cell)

        nbrs = neighbors(cell)

        # Rule 1 / Rule 2 - Breeze  (percept["breeze"] is None => not
observed)
        if percept["breeze"] is True:
            for n in nbrs:
                if n not in self.no_pit:
                    self.possible_pit.add(n)
        elif percept["breeze"] is False:
            for n in nbrs:
                self.no_pit.add(n)
                self.possible_pit.discard(n)

        # Rule 3 / Rule 4 - Stench  (percept["stench"] is None => not
observed)
        if percept["stench"] is True:
            self.stench_sources.append(cell)
            for n in nbrs:
                if n not in self.no_wumpus and n not in
self.confirmed_wumpus:
                    self.possible_wumpus.add(n)
        elif percept["stench"] is False:
            for n in nbrs:
                if n not in self.confirmed_wumpus:
                    self.no_wumpus.add(n)
                    self.possible_wumpus.discard(n)

        self._triangulate_wumpus()
        self._resolve_by_elimination()

        after_safe = set(self.safe_cells())
        newly_safe = sorted(c for c in (after_safe - before_safe) if c !=
cell)
        pit_candidates = sorted(c for c in self.possible_pit if c not in
after_safe)
        wumpus_candidates = sorted(c for c in self.possible_wumpus
                                   if c not in after_safe and c not in
self.confirmed_wumpus)
        return newly_safe, pit_candidates, wumpus_candidates

    def safe_cells(self):
        return ((self.no_pit & self.no_wumpus) | self.visited) -
self.confirmed_wumpus

    def _triangulate_wumpus(self):
        """Two stench cells whose only shared, non-safe neighbour is the
        same cell => that cell must hold the Wumpus (Rule 3 + Rule 5)."""
        sources = self.stench_sources
        safe = self.safe_cells()
        for i in range(len(sources)):
            for j in range(i + 1, len(sources)):
                common = (set(neighbors(sources[i])) &
set(neighbors(sources[j]))) - safe
                if len(common) == 1:
                    self.confirmed_wumpus |= common
                    self.possible_wumpus -= common

    def _resolve_by_elimination(self):
        """If a stench cell has only one neighbour that is still not known
        to be safe, that neighbour must hold the Wumpus (Rule 3 + Rule 5)."""
        safe = self.safe_cells()
```

```
        for source in self.stench_sources:
            unresolved = [n for n in neighbors(source) if n not in safe]
            if len(unresolved) == 1:
                self.confirmed_wumpus.add(unresolved[0])
                self.possible_wumpus.discard(unresolved[0])


def describe(cells):
    return " and ".join(str(c) for c in cells)


def run_agent():
    kb = KnowledgeBase()
    announced_wumpus = set()
    announced_pit_candidates = set()
    announced_wumpus_candidates = set()

    print("Starting position: (1, 1)\n")

    for step, cell in enumerate(PATH):
        percept = PERCEPTS[cell]

        def txt(value, label):
            if value is None:
                return f"{label} not reported"
            return label if value else f"No {label}"

        breeze_txt = txt(percept["breeze"], "Breeze")
        stench_txt = txt(percept["stench"], "Stench")

        print(f"Move: {cell}")
        extra = ", Glitter" if percept["glitter"] else ""
        print(f"Percept at {cell}: {breeze_txt}, {stench_txt}{extra}")

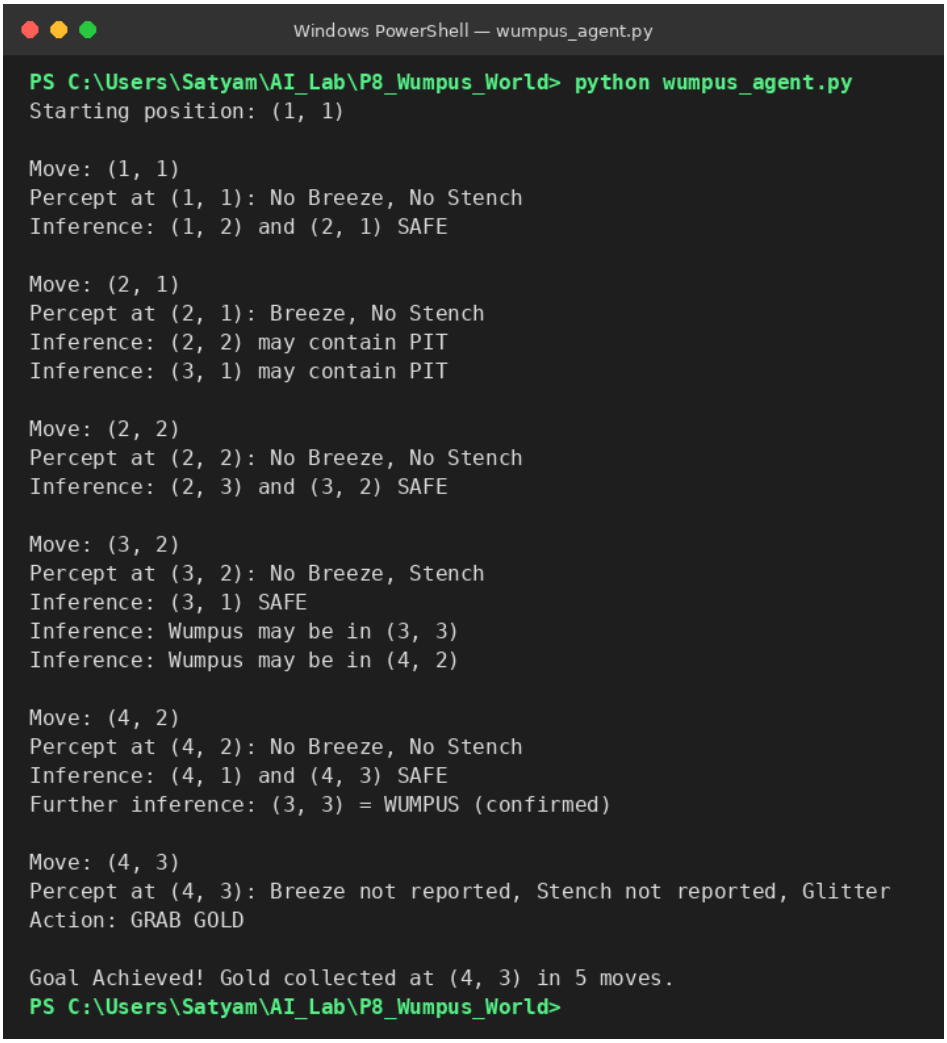
        newly_safe, pit_candidates, wumpus_candidates =
kb.apply_percept(cell, percept)

        if newly_safe:
            print(f"Inference: {describe(newly_safe)} SAFE")
        for c in pit_candidates:
            if c not in announced_pit_candidates:
                print(f"Inference: {c} may contain PIT")
        for c in wumpus_candidates:
            if c not in announced_wumpus_candidates:
                print(f"Inference: Wumpus may be in {c}")
        announced_pit_candidates |= set(pit_candidates)
        announced_wumpus_candidates |= set(wumpus_candidates)

        new_wumpus = kb.confirmed_wumpus - announced_wumpus
        for c in sorted(new_wumpus):
            print(f"Further inference: {c} = WUMPUS (confirmed)")
        announced_wumpus |= kb.confirmed_wumpus

        if percept["glitter"]:
            print("Action: GRAB GOLD")
            print(f"\nGoal Achieved! Gold collected at {cell} in {step}
moves.")
            return

        print()
```

	`if __name__ == "__main__":` `    run_agent()`
output	
Conclusion	In this practical, a knowledge-based agent was designed for the Wumpus World problem by treating the agent's percepts (Breeze, Stench and Glitter) as evidence and applying propositional-logic rules over that evidence rather than assuming complete knowledge of the grid. Starting from (1,1), the agent updated its knowledge base after every move: absence of a breeze or stench immediately marked neighbouring cells safe, while a breeze or stench added the corresponding neighbours as suspected pit or Wumpus locations. Once every other neighbour around a stench source was confirmed safe by direct visitation, the single remaining unresolved neighbour — cell (3,3) — was logically confirmed to contain the Wumpus, purely by elimination, without the agent ever entering that cell. The agent then continued along the frontier of safe cells until it reached (4,3), perceived Glitter, and grabbed the gold in 5 moves. This shows that the Wumpus World is fundamentally a logical-inference problem layered on top of search: an agent that reasons correctly with Rules 1–5 can locate hazards and the

	goal safely even though it never observes the environment directly, which is exactly the behaviour expected of a Knowledge-Based Agent.
Declaration	I have done the practical myself with/ without use of AI tools.